

LOP (Land of Power) 게임 기획서 (GDD)



- 문서 유형: 게임 디자인 문서(GDD)
- 문서 버전: 2.0
- 프로젝트 버전: 1.0.1 (lop/package.json, electron/package.json 기준 — [검증])
- 작성/최신화 기준일: 2026-09-11 (KST)
- 근거 신뢰도: 중간 — 코어 로직(lop/src/lib/*.ts)은 코드 원문을 직접 읽고 확인했으나, UI 세부 문구, 아트 디렉션, 텔레메트리 수치는 코드에 존재하지 않아 대부분 "미검증" 또는 "제안"으로 표기함.

0. 표기 범례

- [검증] = `lop/src/lib/*.ts` , `lop/src/components/*.tsx` , `lop/tests/*` 등 실제 소스 코드를 읽고 확인한 사실.
- [제안] = 코드 근거 없이 기획 목적상 제시하는 아이디어. 실제 구현과 무관.
- 미검증 = 코드/문서 어디에도 해당 수치·정책이 존재하지 않아 확인 불가. 절대 창작하지 않고 이렇게 표기함.

버전 표기 불일치 [검증 — 이슈]

- `lop/package.json` , 루트 `electron/package.json` 모두 `"version": "1.0.1"` .

- 그러나 인게임 시작 화면 `lop/src/components/StartScreen.tsx` 7번째 줄에 `const VERSION = '0.4.21';` 가 하드코딩되어 있고, 화면 우측 상단에 `v{VERSION}` 으로 그대로 노출됨.
- 즉 패키지 버전(1.0.1)과 실제 플레이어가 화면에서 보는 버전(0.4.21)이 다르다. 어느 쪽이 "실제 배포 버전"인지는 코드만으로 확정할 수 없어 미검증으로 남긴다. 다음 우선순위에 수정 항목으로 포함(\$L).

1. 기획 품질 6대 요소

1-1. 문제 정의

[검증] `docs/persona_playtest_feedback.md` (최신화 2026-07-02)는 실제 플레이테스트 페르소나 피드백으로, "선택 결과가 다음 목표와 한 화면에서 강하게 연결되지 않으면 보드 이동이 단순 주사위 결과처럼 느껴질 수 있다"는 문제를 명시하고 있다. [검증] 같은 문서는 "기존 문서가 깨져 있어 게임 방향과 재착수 조건을 읽기 어려웠다"는 문서 신뢰도 문제도 지적한다(이번 GDD 재작성의 직접 동기). [제안] 정리하면 핵심 문제는: (1) 주사위 이동 → 타일 이벤트 → 전투/통행료라는 코어 루프 자체는 구현돼 있으나, 한 행동의 결과가 다음 판단으로 이어지는 인과관계를 플레이어가 화면에서 즉시 읽어내기 어렵다는 것. (2) 캐릭터 간 밸런스가 최종 확정되지 않은 상태(\$9 참조)라는 것.

1-2. 주 페르소나

[검증] `docs/persona_playtest_feedback.md` 에 기재된 실제 테스트 페르소나:

- 이름: 윤하겸, 25세.
- 선호 장르: 짧은 전략 보드게임, 선택형 로그라이크, 캐릭터 성장 게임.
- 플레이 성향: 10~20분 안에 판단과 결과가 이어지는 게임을 선호하며, 선택 결과가 다음 목표로 이어질 때 만족한다.
- 맥락: 캐릭터 성장·병력 구성·다음 장면의 유불리로 이어지는 보드게임을 기대하며 플레이함.

이 인물을 본 문서의 주 페르소나로 채택한다(가명이 아니라 실제 피드백 문서에 등재된 이름).

1-3. 코어 루프 (X→Y→X)

[검증] `lop/src/lib/gameReducer.ts` 의 `TurnPhase` 전이(`gameTypes.ts` 5~19행, `gameReducer.ts` 전체)로부터 확인되는 순환:

플레이어가 주사위를 굴리면(X) 이동 거리가 정해지고, 그 거리만큼 말을 옮겨 **타일 이벤트(전투/구매/통행료/용병/찬스카드)**를 마주하며(Y), 그 결과(점령·패배·후퇴·골드 변화)가 다음 턴의 병력·자금·영토 상태를 바꾸어 다시 주사위를 굴리는 시작점(X) 으로 돌아온다.

세부 턴 상태 머신(TurnPhase , [검증]): `start_deploy → roll → select_piece → (choose_stop 선택적) → tile_event/battle/mercenary/event_card → build/deploy → end_turn → roll`(다음 플레이어 또는 더블 시 같은 플레이어 보너스 턴)

1-4. MVP 가설

[제안] 가설 A: "전투/통행료 결과 화면에 병력 차이·상성·핵심 손실 이유를 한 문장으로 보여주면, 플레이어가 다음 행동을 더 빠르게 결정한다." — `docs/persona_playtest_feedback.md` 의 개선 기준 2번("전투 결과는 병력 차이, 상성, 핵심 손실 이유를 한 문장으로 설명한다")에서 직접 도출. [제안] 가설 B: "선택 결과 화면에 변화 수치와 다음 목표 CTA를 함께 배치하면 세션 이탈이 줄어든다." — 같은 문서 개선 기준 1번에서 도출. 두 가설 모두 검증 방법(A/B 테스트, 텔레메트리)은 코드에 구현되어 있지 않음 — 미검증.

1-5. 참고 분석 (레퍼런스와 교훈)

미검증 — 코드/문서에 참고한 특정 상용 게임 타이틀명이나 그 게임의 단계별 분석이 존재하지 않는다. `docs/LOP_레퍼런스_layout-polish-check-0.4.8.png` 라는 파일이 `docs/` 폴더에 존재하는 것은 확인했으나 [검증: 파일 존재], 그 이미지가 어떤 레퍼런스 게임을 비교한 것인지는 본문(이미지 내용) 확인 없이 추정할 수 없어 창작하지 않는다. [제안] 장르 관례상 "주사위 이동 + 영토 매입/통행료" 구조는 모노폴리 계열 보드게임과 유사하며, 통상적 교훈은 "초반 저비용 타일을 빠르게 확보한 플레이어가 후반 눈덩이 효과를 가진다"는 것이다. 이는 일반적 장르 지식에서 온 제안이며 LOP 자체 데이터로 검증되지 않았다.

1-6. 수치로 측정 가능한 KPI

미검증 — 코드 내에 텔레메트리, 이벤트 로깅 전송, 분석 SDK 연동이 존재하지 않는다(로그는 `state.log:string[]` 로 인메모리 텍스트 로그만 존재, `gameTypes.ts` 147행). [제안, 목표치는 창작 없이 추정만 표기] KPI 후보:

- 첫 주사위 굴림까지 걸리는 시간(스타트 화면 3단계: `goal → setup → reveal` 완료 시간).
- 한 세션(게임오버까지) 평균 소요 턴 수.
- 전투 발생 후 `BATTLE_FINISH` 확인까지의 체류 시간. 목표 수치는 실측 데이터가 없어 제시하지 않음 (창작 금지 원칙 준수).

2. 디자인 필라 (Design Pillars)

[검증 기반 재구성] 코드 구조에서 실제로 관철되는 설계 축:

- 상성 기반 병력 구성 — `battleEngine.ts` 의 `counterMultiplier` 가 병종 간 상성(예: 검사→창병, 군사→검사/창병, 기마병→검사/군사, 창병→기마병)에 최대 +0.6 배율 보너스를 부여([검증], `gameData.ts` 644행 `counters` 필드, `battleEngine.ts` 2233행).

2. **캐릭터별 비대칭 패시브** — 7종 캐릭터가 서로 다른 랩 완주/전투승리 트리거 스킬을 가짐([검증], `gameData.ts` 16~32행).
3. **위협도 있는 이동(패스 지점 정차)** — 이동 중 자신의 영토를 지나칠 때 정차하여 병력을 회수할지, 계속 이동할지 선택하는 `choose_stop` 단계([검증], `gameReducer.ts` 352~359행, `stopAtTileSource.test.mjs`).
4. **경제-군사 이중 압박** — 통행료 미납 시 강제 매각(`forced_sell`)과 파산(`checkBankruptcy`)이 승패에 직결([검증], `gameReducer.ts` 1339~1356행).

3. 컴포넌트 표 (구성요소)

아래 표는 본 게임의 핵심 구성요소를 정리한 것이다.

컴포넌트	역할	플레이어 선택지	입력→판단→피드백	구현 상태
주사위(Roll)	이동 거리 결정	굴리기 버튼 클릭	클릭 → 2d4(각 0~3) 합산, 더블 시 보너스 턴 → 결과 숫자·보너스 턴 로그 표시	구현됨 [검증: <code>gameReducer.ts</code> 543~559행]
말 선택 (SelectPiece)	이동할 유닛 지정	병력이 남은 말 중 택1	클릭 → 목적지 계산 (<code>nextPosition</code>) → 이동 애니메이션	구현됨 [검증: <code>boardLayout.ts</code> 33~35행]
정차 선택 (choose_stop)	지나친 자기 영토 병력 회수 여부	정차/계속 이동	선택 → <code>STOP_AT_TILE</code> 또는 <code>CONTINUE_MOVE</code> → 병력 회수 또는 최종 목적지 도착	구현됨 [검증: <code>gameReducer.ts</code> 352359, 660671행, <code>StopAtTileModal.tsx</code>]
타일 이벤트 (tile_event)	중립/적 영토 도달 시 행동	전투/구매/통행료 납부/패스	상태에 따라 분기 → 전투면 즉시 결과, 구매면 소유권 이전	구현됨 [검증: <code>gameReducer.ts</code> 1166~1198행]
전투(Battle)	공격측· 수비측 병력 소모전	없음(자동 라운드 진행), 전투 전 카드로 공격/수비 배수 결정 가능	진입 → 최대 50라운드 자동 시뮬레이션 → 라운드별 로그 + 승패	구현됨 [검증: <code>battleEngine.ts</code> 45~83행]
배치(Deploy)	점령/구매한 영토에 병력 나눠 배치	병종별 배치 수량 지정	슬라이더/입력 → 배치 확정 → 영토 수비대 (<code>garrison</code>) 갱신	구현됨 [검증: <code>gameReducer.ts</code> 838~861행, <code>DeployModal.tsx</code>]

컴포넌트	역할	플레이어 선택지	입력→판단→피드백	구현 상태
건설(Build)	자기 영토에 건물 건설	금고/병영/요새/ 관문 중 택1, 레벨업	골드 지불 → 건물 레벨 +1 → 통행료/생산/전투 보너스 변화	구현됨 [검증: economyUtils.ts , gameReducer.ts 988~1001행]
상점(Shop)	골드로 병력/유닛 구매	병종별 수량 구매, 신규 말 고용	구매 → 가격은 구매 누적 횟수에 비례해 상승 (troopBuyCount) → 병력 반영	구현됨 [검증: gameReducer.ts 1007~1043행]
용병소 (Mercenary)	랜덤 병종/ 수량 획득	400골드 지불 여부	지불 → 확률적 병종/ 수량 산출(암살자·광전사 포함) → 즉시 말에 합류	구현됨 [검증: gameReducer.ts 593~614행]
찬스 카드 (chance)	이동 중 무작위 이벤트	카드 확인(선택지 없음, 일부는 목적지 선택)	도착 → 25종 카드 중 무작위 1장 적용 → 즉시 효과 반영	구현됨 [검증: gameData.ts 89184행, gameReducer.ts 42041325행]
드래곤 (Dragon)	5랩 후 출현하는 고위험 중립 개체	전투 회피 불가 (조우 시 자동 전투)	조우 → 전투 → 승리 시 골드+800/병력+5, 패배 시 전멸 후퇴	구현됨 [검증: gameReducer.ts 301334, 685719, 13164322, 40671098행]
강제 매각 (ForcedSell)	통행료 부족 시 영토 매각	매각할 영토 선택 (AI는 최저가부터 자동)	매각 → 영토 60% 가치 환급 → 통행료 재정산	구현됨 [검증: gameReducer.ts 920~957행]

4. 콘텐츠 수치 (코드로 검증된 값만 기재)

4-1. 보드

- 총 14칸([검증] boardLayout.ts 28행 TOTAL_TILES = 14).
- 타일 구성: 시작(왕도 성문 0번=player, 변방 요새 7번=ai) 2칸, 땅(land) 8칸(1,2,4,6,8,9,11,13), 찬스(chance) 2칸(5,12), 용병소(mercenary) 2칸(3,10)([검증] boardLayout.ts 11~26, 31행).
- 3인/4인 플레이 시 AI2 시작 4번 칸, AI3 시작 11번 칸([검증] gameReducer.ts 7~8행).
- 땅 티어: 티어1(1,2번), 티어2(4,6,8번), 티어3(9,11,13번)([검증] gameReducer.ts 113행 주석 및 TILE_TIER).
 - 티어별 초기 병력 범위: 티어1 [3,3], 티어2 [3,5], 티어3 [5,7].
 - 티어별 랩 생산 범위: 티어1 [1,2], 티어2 [2,4], 티어3 [3,6].

- 땅값 = 랩생산 × 180, 통행료 기본값 = 랩생산 × 80 (시작 칸 제외)([검증] `gameReducer.ts` 137~138행).
- 시작 칸 초기 병력 12명, 기본 통행료 350([검증] `gameReducer.ts` 120~121행).

4-2. 병종 (6종, `gameData.ts` TROOP_DATA)

병종	공격	방어	가격	상성(강함 대상)	비고
검사 (swordsman)	1.1	1.0	60	창병	상점 구매 가능
궁사(archer)	1.5	0.5	80	검사, 창병	상점 구매 가능
기마병(cavalry)	1.4	0.7	120	검사, 궁사	상점 구매 가능(창병에 약함)
창병(spearman)	0.6	1.6	70	기마병	상점 구매 가능
암살자(assassin)	0.7(말 전투 시 3.8)	0.4	상점 비매(용병소 전용)	상성 없음	말 전투(vsPiece) 전용 고배율
광전사 (berserker)	2.8	0.15	상점 비매(용병소 전용)	상성 없음	초고공격·극저방어

상성 보너스 최대 +0.6배(`COUNTER_BONUS`), 두 진영 구성 비율의 곱으로 가중([검증] `battleEngine.ts` 4, 22~33행).

4-3. 캐릭터 (7종, `gameData.ts` CHARACTERS)

캐릭터	최대 병력	패시브 트리거	효과
선동가(agitator)	40	랩 완주 시	랜덤 병종 2명 합류
흑마술사(warlock)	30	랩 완주 시	적 말 병력 랜덤 2명 제거(저주)
밀수꾼(smuggler)	30	랩 완주 시	골드 +200
사기꾼(swindler)	25	주사위 합계 0(더블 0-0)	골드 +200
종교인(cleric)	40	랩 완주 시	보유 영토에 랜덤 병종 3명 분배
장군(general)	60	전투 승리 시	랜덤 병종 1명 말에 추가(전리품)
해적(pirate)	45	전투 승리 시	골드 +100(약탈)

모든 캐릭터의 기본 공격/방어 배수는 1.0으로 동일하며, 차별화는 최대 병력과 패시브 스킬에서만 발생([검증] `gameData.ts` 25~31행). `moveBonus` 필드는 전 캐릭터 0으로 미사용([검증]).

4-4. 건물 (4종 × 3레벨, `gameData.ts` BUILDING_DATA)

건물	레벨1/2/3 비용	효과(레벨1/2/3)
금고(vault)	200/500/1000	랩당 골드 수입 80/200/400
병영(barracks)	200/500/1000	랩당 병력 생산 +2/+5/+10
요새(fort)	200/500/1000	공격 배수 1.1/1.2/1.35, 방어 배수 1.05/1.15/1.3
관문(toll_gate)	300/750/1500	통행료 전문 강화(자체 공수 보너스 없음)

모든 건물의 레벨별 통행료 기여치도 별도로 존재(`toll` 배열, `economyUtils.ts` 에서 건물 보유 시 최댓값 채택)([검증] `gameData.ts` 4473행, `economyUtils.ts` 413행).

4-5. 경제 상수

- `BASE_LAND_PRICE = 300`, `TROOP_PRICE = 50` (단, 실제 구매가는 `TROOP_DATA[type].price` 사용, 이 상수의 실사용처는 코드 검색 범위 내 미확인 — 미검증), `LAP_TROOP_BONUS = 5` (사용처 미확인 — 미검증), `LAP_GOLD_BONUS = 200` (랩 완주 골드 보너스로 실사용 확인, `gameReducer.ts` 373행), `LAP_LAND_PRODUCTION = 2` (사용처 미확인 — 미검증), `TAX_RATE = 0.1` (`calcTax` 함수는 정의되어 있으나 `gameReducer.ts` 의 리듀서 로직에서 호출부는 확인하지 못함 — 미검증), `TROOP_PRICE_SCALE = 0.06` (상점 구매 누적 시 단가 인상률로 실사용 확인, `gameReducer.ts` 1012행).
- 신규 말 고용가: `[0, 500, 1200, 2000, 5000]`, 이후 `이전값 × 2^n + 300` 로 증가([검증] `gameData.ts` 83~87행).
- 용병소 계약 비용 400골드 고정, 수량은 `floor(r^4 × 7) + 2` (29명), ~~병종은 4종 일반 + 암살자 + 광전사 중 균등 확률 추첨~~([검증] `gameReducer.ts` 596601행).
- 영토 매각가 = 땅값 × 0.6([검증] `gameReducer.ts` 923행).
- 전투 패배 시 통행료를 "배상금" 형태로 영토 소유주에게 지급([검증] `gameReducer.ts` 779~833행).

4-6. 전투 공식

- 데미지 = `ceil(troops × attack × 0.35 / 상대방어)`, 최소 1([검증] `battleEngine.ts` 35~43행 `DAMAGE_RATE = 0.35`).
- 최대 50라운드까지 자동 진행, 그 전에 어느 한쪽 병력이 0 이하가 되면 즉시 종료([검증] `battleEngine.ts` 76~83행).
- 찬스 카드로 공격 1.5배/2배, 방어 1.5배/2배 임시 버프 가능([검증] `gameData.ts` 129~140행).

4-7. 찬스 카드 (25종, `gameData.ts` CHANCE_CARDS)

카테고리별 개수([검증, 배열 직접 카운트]): 즉시 골드 3종(+200/+400/+600), 즉시 병력 3종(+3/+5/+8), 무료·할인 징집 2종, 주사위 보너스 2종(+2/+4), 공격력 버프 2종(1.5배/2배), 방어력 버프 2종(1.5배/2배), 통행료 면제/배가 3종, 건설 할인/무료 2종, 증원(영토 병력) 2종, 드래곤 소환 1종, 이동/특수(임의 이동·상점 이동·랜덤 적 영토 초기화) 3종. 합계 25종.

4-8. 드래곤

- 찬스 카드 `c_dragon` 적용 시 "5바퀴 후" 소환 예약([검증] `gameData.ts` 1701~1722행, `gameReducer.ts` 131~1322행).
- 소환 시 병력 30명, 공격 배수 1.8, 방어 배수 1.2, 소유주가 소환자가 아닌 땅 중 최고가 땅(없으면 전체 최고가 땅)에 출현([검증] `gameReducer.ts` 301302~451464행).
- 매 턴 종료 시 자율적으로 13칸 이동하며 병력이 있는 타일과 자동 교전(1라운드)([검증] `gameReducer.ts` 1069~1098행).
- 플레이어가 드래곤 칸에 도달하면 회피 불가하고 전투 강제 진입([검증] `gameReducer.ts` 1149~1151행).
- 처치 보상: 골드 +800, 랜덤 병종 5명([검증] `gameReducer.ts` 690~704행).

4-9. 승패 조건 (코드 기준)

- 플레이어 골드가 0 미만이면 즉시 게임오버, 승자 AI([검증] `gameReducer.ts` 1340~1341행).
- AI(2,3 포함)는 골드가 0 이하가 되면 개별 파산 처리(영토 중립화, 말 병력 0, 시작 위치 복귀)([검증] `gameReducer.ts` 1327~1350행).
- 모든 AI가 파산하면 플레이어 승리([검증] `gameReducer.ts` 1351~1354행).
- 반면 `StartScreen.tsx` (94~97행)의 UI 카피는 "경제 승리/영토 승리/전투 승리" 3가지를 승리 조건으로 안내하나, 실제 리듀서 코드에는 "영토 승리"나 "전투 승리"를 판정하는 별도 로직이 없다 — **UI 설명과 실제 승패 판정 로직 사이의 불일치**로 기록한다. 실질적으로 구현된 승패 판정은 파산(경제) 조건뿐이다.

4-10. AI 난이도 (3단계, `aiEngine.ts`)

- 쉬움(easy): 말 선택 무작위, 중립 영토는 30% 확률로 패스·나머지는 전투/구매 반반, 통행료는 항상 납부, 건물 건설 안 함, 신규 말 고용 안 함.
- 보통(normal): 병력/비용 비교 후 전투 또는 구매 판단, 골드 600 미만이면 병영 우선 건설, 최대 2기 말 보유, 병력 8 미만 시 검사 구매.
- 어려움(hard): 통행료 부담률(`toll/ai.gold`) 기반 공격 판단 추가, 골드 1000 미만이면 금고 우선, 최대 3기 말 보유, 창병 우선 구매([검증] `aiEngine.ts` 20~166행).

5. 세션 길이별 설명

- **30초** [검증 기반] 스타트 화면 3단계(`goal` → `setup` → `reveal` , `StartScreen.tsx`)를 지나 캐릭터가 확정되고 첫 주사위를 굴리기 직전까지. `beginReveal()` 의 스피ن 연출은 약 2.2초로 고정 타이머 설계 ([검증: `SPIN_DELAYS` 합 약 2065ms + 이후 1900ms 대기, `StartScreen.tsx` 13, 49~50행]). 실제 첫 조작까지 도달 시간의 실측치는 없음(미검증).
- **5분** [제안] 첫 랩(출발점 복귀) 전후, 초기 영토 12칸 확보, ~~1회 이상 전투/통행료 이벤트 경험, 찬스 카드 42장 소진~~ 정도로 추정. 정확한 턴 수는 실측 없음(미검증).
- **30분** [제안] 여러 랩 진행으로 드래곤 출현 조건(랩+5) 충족 가능, 건물 레벨업 1~2회, AI 파산 또는 플레이어 파산 위기 국면 도달 추정. 미검증.
- **장기 세션(엔드게임)** [검증 기반] 승패는 파산 여부로만 결정되므로(\$4-9), 장기적으로는 "누가 먼저 통행료를 못 내고 영토를 매각하다 자금이 바닥나는가"의 경제 소모전으로 수렴하는 구조. 이는 코드 로직에서 직접 도출되는 필연적 귀결이다.

6. 재미 사슬 (Fun Chain)

[검증 기반 재구성, 일부 인과관계는 코드 구조에서 합리적으로 추론한 것으로 [제안] 표기]

1. **재미 요소:** 상성을 활용한 역전 가능성(약자도 상성으로 이길 수 있음).
2. **트리거:** 중립 또는 적 영토에 도달해 `tile_event` 상태 진입([검증] `handleTileLanding`).
3. **플레이어 판단/행동:** 현재 병력 구성과 상대 수비대 구성을 비교해 전투(`CHOOSE_FIGHT`) 또는 회피(통행료 납부/패스)를 선택.
4. **즉각 피드백:** 전투 시 라운드별 로그(`BattleRound.log` , "공격 N명 → 적 M명 전사 / 수비 N명 → 아군 M명 전사")로 매 라운드 결과 확인([검증] `battleEngine.ts` 56행), 최종적으로 영토 점령(기본 5명 자동 배치) 또는 후퇴 로그 표시.
5. **반복 동기:** 승리 시 점령한 영토에 추가 배치할 병력 배분 결정(`deploy` 단계)이 다음 랩의 생산량과 방어력을 좌우하므로, 다음 턴에도 유사한 판단을 반복하도록 유도됨. 더블 주사위로 얻는 보너스 턴 ([검증] `gameReducer.ts` 549, 1100~1116행)은 같은 루프를 즉시 재실행하게 만드는 직접적 반복 장치.

7. 실제 플레이 예시 (코드 로직 기반, 창작 없이 논리적으로 구성)

예시 1 — 초반 중립 영토 점령

1. 플레이어가 `roll` 단계에서 주사위를 굴려 3이 나옴(2d4, 각 03-범위이므로 ~~실제 합은 06~~ 사이 정수) ([검증] `gameReducer.ts` 547~550행).
2. `select_piece` 에서 유일한 말을 선택, 3칸 이동해 티어1 땅(예: 1번 "드래곤 고원")에 도달.
3. 해당 땅이 중립이고 병력 3명이면(`tier=1` 범위 [3,3]) `tile_event` 로 진입, 플레이어는 땅값(생산량×180)을 확인하고 구매 또는 전투를 선택.

4. 구매를 선택하면 즉시 소유권을 얻고 `deploy` 단계로 진입해 자신의 말 병력 일부를 배치([검증] `gameReducer.ts` 863~879행).
5. 다음 랩 완주 시 이 땅에서 티어1 생산량(1~2명) + 건물 생산이 자동으로 증가함([검증] `resolvePieceMove` 랩 완주 처리부).

예시 2 — 드래곤 조우와 보상

1. 이전 턴에 뽑은 찬스 카드가 `c_dragon` 이었고, "5바퀴 후 드래곤 출현" 예약이 걸림([검증] `gameData.ts` 170~172행).
2. 5랩 경과 시점에 소유주가 다른(또는 중립) 최고가 땅에 드래곤(병력 30, 공격1.8/방어1.2)이 출현([검증] `gameReducer.ts` 451~464행).
3. 플레이어의 말이 이동 중 우연히 드래곤이 있는 칸에 도착하면 회피 불가하고 즉시 전투가 시작됨([검증] `executeDragonBattle` , `gameReducer.ts` 1149~1151행).
4. 전투에서 승리하면 골드 +800과 랜덤 병종 5명을 즉시 획득하고 드래곤은 사라짐. 패배하면 병력 전멸 후 가장 가까운 소유 영토(또는 시작 칸)로 후퇴([검증] `gameReducer.ts` 686~719행).

두 예시 모두 코드에 존재하는 값과 분기만 사용했으며, 대사·연출 세부는 창작하지 않았다.

8. 피로/실패 요소 및 완화 방안

[검증된 실패 지점]

- 통행료를 낼 골드가 없으면 강제 매각(`forced_sell`) 단계로 강제 진입([검증] `gameReducer.ts` 881~896행). 매각할 땅이 없거나 매각해도 부족하면 골드가 음수로 내려가 파산 처리([검증] `checkBankruptcy`).
- 전투 패배 시 병력 전멸 + 통행료(배상금) 이중 손실 발생([검증] `gameReducer.ts` 722~835행) — 공격적 플레이어의 리스크가 명확히 큼.
- 최대 50라운드 전투 자동 진행 중에는 플레이어가 개입할 수 없음([검증] `runFullBattle`) — 결과를 지켜보기만 하는 구간이 길어질 경우 피로 요소가 될 수 있음(제안: 라운드 스킵 버튼 등, 미구현).

[완화 방안 — 제안, 미구현]

- 강제 매각 전 "위험 임계 골드" 경고 배너로 사전 대비 유도.
- 전투 자동 진행 애니메이션에 스킵/배속 옵션 제공.
- `docs/persona_playtest_feedback.md` 개선 기준에 따라 전투 결과 요약 한 문장(병력차·상성·손실 이유) 표시.

9. 경제/성장/밸런스 레버

[검증된 레버]

- 상점 구매가 인상: 구매할 때마다 `troopBuyCount` 가 누적되어 단가가 $1 + \text{count} \times 0.06$ 배로 상승([검증] `gameReducer.ts` 1007~1026행) — 과도한 병력 매집을 억제하는 레버.
- 신규 말 고용가 급상승 곡선(`nextHireCost` , 500→1200→2000→5000→이후 2배+300)([검증] `gameData.ts` 84~87행) — 다중 말 운용을 후반으로 갈수록 비싸게 만드는 레버.
- 건물 레벨업 비용은 선형이 아니라 200→500→1000(또는 300→750→1500)으로 체감 이득이 줄어드는 구조([검증] `gameData.ts` 44~73행).
- 통행료는 랩 수에 비례해 자동 인상(`lapBonus = 1 + lapCount \times 0.12`)([검증] `economyUtils.ts` 5행) — 게임 후반으로 갈수록 영토 미보유 플레이어의 부담이 커지는 스노우볼 레버.
- 캐릭터별 최대 병력 상한(25~60)이 상이해 공격적 확장/수비 전략의 상한을 다르게 제한([검증] `gameData.ts` CHARACTERS).

[미검증/제안] 각 레버의 최종 수치가 실제 플레이 밸런스에서 "적정"한지에 대한 실측 데이터(승률 통계 등)는 존재하지 않음. `docs/LOP_업데이트_내역서.md` 의 "남은 작업" 항목에도 "캐릭터 밸런스 미가입 항목 확정"이 명시되어 있어([검증]), 밸런스가 아직 확정 단계가 아님을 시사한다.

10. 온보딩 / UI-HUD / 접근성 / 오디오비주얼

10-1. 온보딩 (검증됨)

`StartScreen.tsx` 기준 3단계 온보딩 화면 흐름([검증]):

1. `goal` — 게임 목표/승리 조건 설명, 3단계 요약("굴리기/점령하기/키우기").
2. `setup` — 참가 인원(2/3/4명), 난이도(쉬움/보통/어려움) 선택.
3. `reveal` — 캐릭터 랜덤 추첨 연출 후 게임 시작.

10-2. HUD/UI 상태 (5가지, 코드 존재 기준)

테스트 파일명과 상태 로직으로부터 확인되는 5개의 대표 UI/턴 상태([검증: `turnActionInfo.mjs` 가 정의하는 `phase→step/title/description/tone` 매핑, 총 12개 phase 항목 존재):

1. 배치 상태(`deploy` , `start_deploy`) — "배치" 톤 `ready` .
2. 타일 이벤트/조우 상태(`tile_event`) — "3단계, 타일 행동".
3. 정차 선택 상태(`choose_stop`) — "경유지 정차".
4. 전투 상태(`battle`) — 톤 `danger` .
5. 위기(강제 매각) 상태(`forced_sell`) — 톤 `danger` , "통행료 정산".

(실제로는 `turnActionInfo.mjs` 에 12개 phase 매핑이 모두 정의되어 있음 — [검증]. 위 5가지는 GDD 요구 형식에 맞춰 대표 상태만 발췌한 것이며, 전체 목록은 `roll, select_piece, choose_move_tile, choose_stop,`

tile_event, battle, deploy, build, shop, mercenary, event_card, forced_sell, end_turn 이다.)

10-3. 로딩/빈 상태/에러/대량 데이터/긴 텍스트 대응

미검증 — 코드 내에 별도의 로딩 스피너 상태, 빈 데이터 플레이스홀더, 에러 바운더리, 100개 이상 데이터 리스트 가상화, 3배 긴 텍스트 대응 로직이 확인되지 않았다. 보드가 14칸 고정이라 "100개 이상 데이터" 시나리오 자체가 현재 스코프에 해당하지 않을 가능성이 높으나, 이는 추정이며 창작하지 않는다.

10-4. 접근성

미검증 — 코드에서 ARIA 속성, 키보드 전용 내비게이션, 색맹 모드, 자막/캡션 등 접근성 기능이 확인되지 않았다.

10-5. 시각/청각/햅틱 피드백

[검증] 시각 리소스 존재: docs/LOP_gameplay_preview.png (콘셉트 이미지, 실제 게임 화면 아님 — 기존 문서 표기 보존), docs/splash.png , docs/app_icon.png , lop/public 내 캐릭터/배경 이미지(CHAR_IMAGE , charImages.ts 로 참조 확인), 시작 화면 배경 /generated/lop-start-hero-bg.png ([검증] StartScreen.tsx 8행). [검증] lop/tests/game-audio-layer.test.cjs , gameAudioLayer.test.cjs 테스트 파일 존재로 오디오 레이어 관련 코드가 존재함을 시사하나, 테스트 본문 상세와 실제 사운드 자산 목록까지는 이번 조사 범위에서 전부 확인하지 못함 — 세부 트리거 조건은 미검증. 햅틱 피드백: 미검증(데스크톱 Electron 앱 특성상 해당 사항 없을 가능성 높으나 단정하지 않음).

11. 구현됨 / 계획됨 / 제안 구분

구현됨 [검증]

- 14칸 보드, 턴제 주사위 이동(2d4), 더블 보너스 턴.
- 6개 병종 + 상성 시스템, 7개 캐릭터 패시브.
- 4종 건물(3레벨), 통행료/땅값/랩 생산 공식.
- 전투 시뮬레이션(라운드제, 최대 50라운드), 패배 시 후퇴 로직(시작 기지 또는 최근점 소유 영토).
- 25종 찬스 카드, 용병소, 강제 매각, 파산 기반 승패 판정.
- 드래곤 이벤트(소환 예약→출현→자율 이동→처치 보상).
- 2/3/4인 플레이, 3단계 AI 난이도.
- 스타트 화면 3단계 온보딩, 12개 턴 상태에 대한 UI 텍스트 매핑(turnActionInfo.mjs).
- Electron 데스크톱 포터블 빌드, Steam 연동 옵션(steamworks.js , optionalDependencies)([검증] lop/package.json).

계획됨 (문서상 명시된 미완료 작업) [검증]

- docs/LOP_업데이트_내역서.md "남은 작업" 항목: 캐릭터 밸런스 미가입 항목 확정, 전투 결과 설명의 납득도 플레이테스트, dirty 빌드 산출물과 release 상태 정리, Electron package metadata UTF-8/ASCII 안전화.
- docs/persona_playtest_feedback.md "다음 개선 기준": 선택 결과 화면에 변화 수치·다음 목표 CTA 배치, 전투 결과 한 문장 설명, 캐릭터 밸런스 보류 상태 재평가.

제안 (본 문서에서 새로 제시, 미구현) [제안]

- "영토 승리/전투 승리" UI 카피와 실제 파산 전용 승패 로직의 불일치 해소(\$4-9).
- 인게임 표시 버전(0.4.21)과 패키지 버전(1.0.1) 일치화(\$0).
- 전투 자동 진행 스킵/배속 옵션.
- 강제 매각 이전 위험 임계 경고 UI.

11-1. 콘텐츠 제공 방식

[검증 기반 요약 — 새로운 메커닉을 추가하지 않고 위 \$4, \$11에 이미 기재된 검증 내용만 요약함]

본 게임의 콘텐츠는 아래와 같은 방식으로 플레이어에게 제공된다:

- 해금:** 드래곤은 찬스 카드 `c_dragon` 적용 후 "5랩 후" 조건이 충족되어야 해금(출현)된다(\$4-8). 신규 말 고용, 건물 레벨업(1→2→3) 역시 골드 조건을 충족해야 순차적으로 해금된다(\$4-4, \$4-5).
- 순서:** 콘텐츠는 `roll` → `select_piece` → (`choose_stop`) → `tile_event/battle/mercenary/event_card` → `build/deploy` → `end_turn` 의 턴 순서를 따라 제공되며(\$1-3), 랩을 반복할수록 티어가 높은 땅·드래곤 조우 같은 후반 콘텐츠가 순차적으로 노출된다(\$4-1, \$4-8).
- 보상:** 전투 승리 시 영토 점령 및 캐릭터별 패시브 보상(\$4-3), 드래곤 처치 시 골드 +800·병종 5명(\$4-8), 랩 완주 시 생산량 증가(\$4-1)와 같은 보상이 지급된다.
- 변형:** 찬스 카드 25종(\$4-7), 캐릭터 7종의 서로 다른 패시브(\$4-3), AI 난이도 3단계(\$4-10)가 매 판마다 다른 변형된 진행 경험을 만든다.

성공 KPI [제안]

[제안 — 실측 텔레메트리가 코드에 없으므로(\$1-6) 아래 항목은 모두 제안이며 목표 수치는 미기재. \$1-6, \$1-4 KPI 후보를 기반으로 확장]

- 첫 주사위 굴림 도달률/시간** — 온보딩 3단계(`goal` → `setup` → `reveal`) 완료 후 첫 `roll` 실행까지의 시간·이탈률.
- 세션 완주율** — 30초 진입 세션 대비 5분·30분 세션까지 도달하는 플레이어 비율(\$5).

3. 전투 개입 빈도 — 세션당 `tile_event` 에서 전투를 선택하는 비율 대 통행료 납부/패스를 선택하는 비율.
4. 파산 발생 시점 분포 — `checkBankruptcy` 가 트리거되는 평균 턴 수(장기 세션 길이 추정 지표, §4-9, §5).
5. 드래곤 조우/처치율 — 5랩 이상 도달한 세션 중 드래곤과 조우 및 처치에 성공하는 비율(§4-8).

12. 다음 우선순위 3가지

1. 버전 표기 불일치 해결: `StartScreen.tsx` 의 하드코딩된 `VERSION = '0.4.21'` 을 `package.json` 의 실제 버전(1.0.1 또는 그 후속)과 동기화하는 절차를 만든다. 현재는 수동 관리로 인해 이미 어긋나 있음([검증] §0).
2. 승리 조건 문구와 실제 판정 로직 일치화: `StartScreen.tsx` 가 안내하는 "영토 승리/전투 승리"를 실제로 구현하거나, 안내 문구를 파산 기반 판정 하나로 정정한다(§4-9).
3. 전투/선택 결과 피드백 강화: `docs/persona_playtest_feedback.md` 가 지목한 대로 전투 결과에 병력차·상성·손실 이유를 한 문장으로 요약하고, 선택 결과 화면에 변화 수치와 다음 목표 CTA를 함께 배치한다 (§8, §11).

13. 근거 노트 (Evidence Note)

이번 작성 시 직접 열람하여 근거로 삼은 실제 구현 파일:

- `C:\Development\3_LOP\lop\package.json` , `C:\Development\3_LOP\electron\package.json` — 버전 정보.
- `C:\Development\3_LOP\lop\src\lib\gameTypes.ts` — 상태/타입 정의.
- `C:\Development\3_LOP\lop\src\lib\gameData.ts` — 병종/캐릭터/건물/찬스카드 수치.
- `C:\Development\3_LOP\lop\src\lib\battleEngine.ts` — 전투 데미지/상성 공식.
- `C:\Development\3_LOP\lop\src\lib\economyUtils.ts` — 통행료/생산/건설비 계산.
- `C:\Development\3_LOP\lop\src\lib\boardLayout.ts` — 보드 타일 정의.
- `C:\Development\3_LOP\lop\src\lib\gameReducer.ts` — 턴 진행, 전투 결과 처리, 드래곤, 파산 판정 등 전체 게임 로직.
- `C:\Development\3_LOP\lop\src\lib\aiEngine.ts` — 3단계 AI 난이도 로직.
- `C:\Development\3_LOP\lop\src\lib\turnActionInfo.mjs` — 턴 상태별 UI 텍스트 매핑.
- `C:\Development\3_LOP\lop\src\components\StartScreen.tsx` — 온보딩 흐름, 인게임 버전 표기, 캐릭터 소개 카피, 승리 조건 카피.
- `C:\Development\3_LOP\lop\tests\defeatRetreatSource.test.mjs` , `stopAtTileSource.test.mjs` (내용 직접 확인) 및 `deployModalSource.test.mjs` , `tileEncounterSource.test.mjs` , `turnActionInfo.test.mjs` , `uxPolishSource.test.mjs` , `imageResources.test.mjs` , `goldAmountSource.test.mjs` , `uiTypographySource.test.mjs` , `game-audio-layer.test.cjs` , `gameAudioLayer.test.cjs` (파일 존재 확인, 두 건은 내용까지 직접 확인해 "소스 코드 정적 패턴

검사"임을 확인 — 런타임 동작을 실제로 실행해 검증하는 테스트가 아니라, 소스 문자열에 특정 패턴이 존재하는지 정규식으로 확인하는 방식임에 유의).

- `C:\Development\3_LOP\docs\persona_playtest_feedback.md` — 실제 페르소나(윤하겸), 문제 정의, 개선 기준.
- `C:\Development\3_LOP\docs\LOP_업데이트_내역서.md` — 남은 작업 목록.
- `C:\Development\3_LOP\docs\LOP_GDD.md` (기존본, 개정 전) — 이전 문서 버전 확인용.

이미지 `LOP_gameplay_preview.png` 는 GDD용 콘셉트 이미지이며 실제 게임 화면이 아니라는 기존 문서의 설명을 그대로 보존한다(\$10-5).

검증(코드 확인)과 제안(디자인 아이디어)은 본문 전체에 걸쳐 `[검증]` / `[제안]` / "미검증" 표기로 구분했으며, 표기가 없는 서술형 문장은 해당 문단 앞뒤의 표기를 따른다.